3.6、图像梯度

[OpenCV: Image Gradients]
https://docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html

1. 什么是图像梯度?

简单来说：

- 图像梯度就是图像中像素值变化的剧烈程度。
- 变化大的地方（从黑到白、从亮到暗）通常就是边缘。
- 一阶导数（Sobel/Scharr）可以找到边缘的方向和强度。
- 二阶导数（Laplacian）可以找到边缘的位置（变化最快的地方）。

OpenCV 提供了三种常用的高通滤波器（High-pass filters）来计算梯度：

- Sobel：最常用，结合了平滑和求导，对噪声有一定抵抗力。
- Scharr：Sobel 的改进版，尤其在求梯度方向时更准确（3x3 时比 Sobel 好）。
- Laplacian：二阶导数，对噪声更敏感，但能直接突出边缘。

2. 核心函数介绍

(1) Sobel 算子

```
cv.Sobel(src, ddepth, dx, dy, ksize=3, ...)
```

- src：输入图像（通常是灰度图）
- ddepth：输出图像的数据类型（深度）。常用 cv.CV_64F（64位浮点数），因为梯度有正负值。
- dx：x方向（水平）的导数阶数（0或1）
- dy：y方向（垂直）的导数阶数（0或1）
- ksize：核的大小（奇数，如 3、5、7）。如果设为 -1，则使用 3x3 的 Scharr 滤波器。=

(2) Scharr 算子

```
cv.Scharr(src, ddepth, dx, dy, ...)
```

- 专门用于 3x3 核，比 Sobel 的 3x3 更精确（权重设计更好）。

(3) Laplacian 算子

```
cv.Laplacian(src, ddepth, ksize=1, ...)
```

- 计算二阶导数：$\Delta = \partial^2/\partial x^2 + \partial^2/\partial y^2$
- 常用 1x1 或 3x3 核。

3. 重要注意事项（教程里强调的“一个重要问题”）

当你直接用 cv.CV_8U（即 uint8，0-255）作为输出类型时：

- 正梯度（黑→白）正常显示
- 负梯度（白→黑）会被截断为 0，导致丢失一半的边缘！
  正确做法：

1. 用高精度类型（如 cv.CV_64F）计算
2. 取绝对值 np.absolute()
3. 再转回 uint8 显示

4. 完整代码示例

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

# 读取灰度图像
img = cv.imread('dave.jpg', cv.IMREAD_GRAYSCALE)   # 换成你自己的图片路径
if img is None:
    print("图片读取失败，请检查路径")
    exit()

# 计算各种梯度
laplacian = cv.Laplacian(img, cv.CV_64F)                    # 二阶导数
sobelx = cv.Sobel(img, cv.CV_64F, 1, 0, ksize=5)           # 水平方向（x）
sobely = cv.Sobel(img, cv.CV_64F, 0, 1, ksize=5)           # 垂直方向（y）

# 显示结果
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1), plt.imshow(img, cmap='gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])

plt.subplot(2, 2, 2), plt.imshow(laplacian, cmap='gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
```

```
plt.subplot(2, 2, 3), plt.imshow(sobelx, cmap='gray')
plt.title('Sobel X (水平)'), plt.xticks([]), plt.yticks([])

plt.subplot(2, 2, 4), plt.imshow(sobely, cmap='gray')
plt.title('Sobel Y (垂直)'), plt.xticks([]), plt.yticks([])

plt.show()
```

5. 实际建议

- 大多数情况推荐用 ksize=3 或 5，ddepth=cv.CV_64F，最后取绝对值。
- 想更精确的水平/垂直梯度，可以用 cv.Scharr() 代替 cv.Sobel(..., ksize=3)。
- Laplacian 对噪声敏感，通常先用高斯模糊 cv.GaussianBlur() 再做。
- 梯度幅度（强度）可以这样计算：gradient_magnitude = np.sqrt(sobelx^2 + sobely^2)
- 梯度方向（角度）常用于后续的 Canny 边缘检测或 HOG 特征。

6.Sobel 和 Laplacian 在图像上到底做了什么

把图像看成一张亮度高度图（像素值 = 海拔高度），Sobel 和 Laplacian 就是用导数来“测量地形变化”，从而突出边缘。

1. Sobel 算子做了什么?

Sobel 是 一阶导数 的近似计算器。

- 它计算的是：图像在水平（x方向） 和 垂直（y方向） 上像素亮度变化的速度（斜率）。
  - SobelX（dx=1, dy=0）：测量左右方向的亮度变化 → 检测垂直边缘（比如竖直的线条、物体的左右边界）。
  - SobelY（dx=0, dy=1）：测量上下方向的亮度变化 → 检测水平边缘（比如横向的线条、物体的上下边界）。
- 具体操作过程：
  1. 先做一点轻微平滑（内置类似高斯平滑的权重），减少噪声影响。
  2. 用一个 3x3（或更大）的卷积核在图像上滑动。
  3. 核会计算中心像素与周围像素的加权差值，得到梯度值（正值或负值）。
     - 正值：亮度从暗→亮（黑→白）
     - 负值：亮度从亮→暗（白→黑）
- 结果图像的特点：
  - 边缘处数值很大（亮度变化快的地方很亮）。
  - 平坦区域数值接近 0（几乎黑色）。
  - 可以分别得到方向信息（水平梯度大还是垂直梯度大）。

- 边缘通常比较粗（因为斜率大的区域有一段宽度）。
  总结一句话：
  Sobel 用一阶导数告诉你“这里坡度有多陡”，从而找出边缘的强度和大致方向。它对噪声有一定抵抗力，是实际项目中最常用的梯度计算方式。

2. Laplacian 算子做了什么？

Laplacian 是 二阶导数 的计算器（它内部其实也用了 Sobel 来计算二阶）。

- 它计算的是：图像亮度变化的加速度（斜率本身的变化率），也就是 $\Delta I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$（x 和 y 方向二阶导数的和）。
- 具体操作过程： 用一个小的卷积核（常用 1 或 3）直接计算二阶差分。
  - 在边缘的中心位置，二阶导数会穿过零（从正变负，或从负变正）。
  - 边缘两侧会出现正值和负值（双边响应）。
- 结果图像的特点：
  - 边缘处出现明显的零交叉（正负跳变的地方就是边缘最精确的位置）。
  - 能检测所有方向的边缘（各向同性，不分水平垂直）。
  - 边缘非常细锐，但也容易把噪声（小颗粒）误认为是边缘，导致结果比较“脏”。
    总结一句话：
    Laplacian 用二阶导数告诉你“坡度在哪里突然改变”（即转折点），从而精确定位边缘的位置。它对细小细节和弱边缘更敏感，但怕噪声。

3. Sobel 和 Laplacian 的直观对比

- Sobel（一阶）：
  - 像测量“坡度大小” → 边缘粗、连续、有方向。
  - 优点：抗噪能力较好，能得到梯度幅度和方向（后面 Canny 就用它）。
  - 缺点：边缘定位不够精确（边缘有点“胖”）。
- Laplacian（二阶）：
  - 像测量“坡度变化的位置” → 边缘细、锐利、无方向。
  - 优点：边缘位置更准确。
  - 缺点：对噪声非常敏感（实际用时通常先 GaussianBlur 再 Laplacian，称为 LoG）。

4. 它们共同的目标两者都是高通滤波器

作用是抑制平坦区域（低频），突出变化剧烈的地方（高频 = 边缘）。

- Sobel 更实用（方向 + 强度）。
- Laplacian 更“数学纯粹”（精确位置），但需要配合去噪。